Data Lab Solution Docs

笃实书院46班 谭承越 学号2024013405

目录

1. bitXor - 异或运算 (Rating: 1)
2. tmin - 最小补码整数 (Rating: 1)
3. isTmax - 判断是否最大补码数 (Rating: 1)
4. allOddBits - 判断所有奇数位是否为1 (Rating: 2)
5. negate - 取反运算 (Rating: 2)
6. isAsciiDigit - 判断ASCII数字字符 (Rating: 3)
7. conditional - 条件运算 (Rating: 3)
8. isLessOrEqual - 比较大小 (Rating: 3)
9. logicalNeg - 逻辑非运算 (Rating: 4)
10. howManyBits - 求补码表示所需最小位数 (Rating: 4)
11. floatScale2 - 浮点数乘2 (Rating: 4)
12. floatFloat2Int - 浮点数转整数 (Rating: 4)
13. floatPower2 - 2.0^x (Rating: 4)

---

1. bitXor

题目描述

实现 `x ^ y`，但只能使用 `~` 和 `&` 操作符。

实现代码

```
int bitXor(int x, int y) {
  /*
  100 ^ 101 == 001
  a^b = ~((~(a & ~b)) & (~(~a & b)))
  */
  return ~((~(x & ~y)) & (~(~x & y)));
}
```

总操作数: 8 ops (max: 14)

解题思路

突破口/关键点:

- 利用德摩根定律将异或拆解为与非和或非的组合
- 核心恒等式: `a ^ b = ~((~(a & ~b)) & (~(~a & b)))`

易错点:

- 不能直接使用 `^`，必须完全用 `~` 和 `&` 表达
- 注意括号的正确性，确保运算顺序正确

---

2. tmin

题目描述

返回补码表示的最小整数 (0x80000000)。

实现代码

```
int tmin(void) {
  return  1 << 31;
}
```

总操作数: 1 ops (max: 4)

解题思路

突破口/关键点:

- 补码最小值的二进制表示是符号位为1，其余位为0
- 将 `1` 左移 31 位得到 `0x80000000`

易错点:

- 注意是 `1 << 31` 而不是 `1 << 32`（移位超过31位是未定义行为）
- 返回类型是 `int`，不是 `unsigned`

---

3. isTmax

题目描述

判断 `x` 是否为补码最大整数 (0x7FFFFFFF)，是返回1，否则返回0。

实现代码

```c
int isTmax(int x) {
  /* //利用Tmax的性质: TMax + 1 = TMin, TMax ^ TMin = -1 (全1)
  //可以通过 !(~z) 将 全1/非全1 转化为 1/0
  //需要注意的是, 满足上式的还有 -1 + 1 = 0, -1 ^ 0 = -1 (全1)
  //怎么证明满足这个条件的只有这两种情况? (一个数+1之后的结果和自己按位完全相反) */
  int y = x + 1;
  return !(~(x ^ y)) & !!y;
}
```

总操作数: 7 ops (max: 10)

解题思路

突破口/关键点:

- TMax = 0x7FFFFFFF，其性质：`TMax + 1 = TMin = 0x80000000`
- `TMax ^ TMin = 0xFFFFFFFF` （全1）
- 利用 `!(~(x ^ y))` 将"全1"转化为1，"非全1"转化为0

关键发现:

- 满足 `x + 1 = ~x` （即 `x ^ (x+1) = -1`）的数只有 TMax 和 -1
- 但 -1 不满足题目条件，所以需要排除它
- -1 的特征是 `y = x + 1 = 0`，因此加 `!!y` 条件排除

证明:

- `~x = -x - 1`，根据异或的性质，将 `x ^ (x+1) = -1` 转化为：`x + 1 = ~x`
- 代入恒等式：`x + 1 = -x - 1`
- 移项得：`2x = -2`
- 即 `2x ≡ -2 (mod 2^32)`
- 在模 2^{32} 的算术系统中，方程 $2x \equiv -2 \pmod{2^{32}}$ 的解为：
  - **解 A：** $x = 2^{31} - 1 = T_{max}$ (`0x7FFFFFFF`)
  - **解 B：** $x = 2^{32} - 1 = -1$ (`0xFFFFFFFF`)

易错点:

- 容易忽略 -1 也满足 `x ^ (x+1) = -1` 的性质
- 必须排除 x = -1 的情况

---

4. allOddBits

题目描述

判断 `x` 的所有奇数位（0,2,4,...,30）是否全为1，是返回1，否则返回0。

实现代码

```
int allOddBits(int x) {
  int mask = 0xAA;
  mask = mask | (mask << 8);
  mask = mask | (mask << 16);
  return !(mask ^ (x & mask));
  /* // int y = x & mask;
  // return !~(y | (y >> 1)) */
}
```

总操作数: 7 ops (max: 12)

解题思路

突破口/关键点:

- 构建奇数位全为1的掩码 `0xAAAAAAAA`
- 分步构建：0xAA → 0xAAAA → 0xAAAAAAAA
- 用 `x & mask` 提取奇数位，比较是否等于 mask

易错点:

- 掩码构建时注意使用 `|` 而不是 `+`（避免进位问题）
- 另一种方法 `return !~(y | (y >> 1))` 利用了"奇数位为1则奇数位|偶数位全1"的性质

---

5. negate

题目描述

返回 `-x`（即 `x` 的加法逆元）。

实现代码

```
int negate(int x) {
  return ~x + 1;
}
```

总操作数: 2 ops (max: 5)

解题思路

突破口/关键点:

- 补码的加法逆元 = 按位取反 + 1
- 这是补码的基本性质：`x + (~x + 1) = 0`

易错点:

- 记住是 `~x + 1` 而不是 `~x` 或其他形式
- 注意 `-TMin` 的结果仍是 `TMin`（溢出）

---

6. isAsciiDigit

题目描述

判断 `x` 是否在 ASCII '0' 到 '9' 之间（即 0x30 <= x <= 0x39）。

实现代码

```
int isAsciiDigit(int x) {
  int a = (x ^ 0x30) >> 3;
  int b = (x ^ 0x38) >> 1;
  return (!a) | (!b);
}
```

总操作数: 7 ops (max: 15)

解题思路

突破口/关键点:

- 将问题转化为判断 x 是否在 [0x30, 0x39] 范围内
- 利用掩码和移位技巧高效判断：
  - `x ^ 0x30` 高位为0表示 x ∈ [0x30, 0x3F]
  - `x ^ 0x38` 高位为0表示 x ∈ [0x30, 0x39] 或 x ∈ [0x38, 0x3F]

- 同时满足两个条件才能确定在 [0x30, 0x39] 内

易错点:

- 不能用减法（减法可能溢出）
- 注意位运算的巧妙应用，避免逐位比较

---

7. conditional

题目描述

实现三元条件运算符 `x ? y : z`。

实现代码

```
int conditional(int x, int y, int z) {
  /* //思路:
  //先想到将conditional转化成(y & y_mask) | (z & z_mask)的形式
  //x == 0则 y_mask = 0, z_mask = 11..1, etc
  //再来想mask怎么用x来表示 */
  int y_mask = ~(!x) + ((!x) << 1); /* //可以直接用~z_mask */
  int z_mask = ~(!x) + 1;
  return (y & y_mask) | (z & z_mask);
}
```

总操作数: 11 ops (max: 16)

解题思路

突破口/关键点:

- 将条件表达式转化为 `(y & y_mask) | (z & z_mask)` 的形式
- 当 x 为真时：y_mask = 0xFFFFFFFF, z_mask = 0x00000000
- 当 x 为假时：y_mask = 0x00000000, z_mask = 0xFFFFFFFF
- 利用 `!x` 的特性（x≠0 时为0，x=0 时为1），结合移位构建掩码

易错点:

- `~(!x)` 将 0 变成 -1 (0xFFFFFFFF)，将 1 变成 -2 (0xFFFFFFFE)

---

8. isLessOrEqual

题目描述

判断是否有 `x <= y`，是返回1，否则返回0。

实现代码

```
int isLessOrEqual(int x, int y) {
  int cond = !(x^y); /* //equal */
  int sign_x = (x >> 31) & 1;
  int sign_y = (y >> 31) & 1;
  int diff_sign = (sign_x ^ sign_y);
  int diff = x + (~y) + 1; /* //注意正溢出问题 */
  cond = cond | (diff_sign & sign_x); /* //如果x,y符号相反，且x为负数，则return 1; x为正数,则return 0; */

  cond = cond | (!diff_sign & !!((diff >> 31) & 1)); /* //需要same_sign，在这里起到shortcut的作用！确保在符号不同时，不进行这一步 */
  return cond;
}
```

总操作数: 19 ops (max: 24)

解题思路

突破口/关键点:

- 分三种情况考虑：
  1. `x == y`：直接返回1
  2. `x` 和 `y` 符号不同：负数小于正数
  3. `x` 和 `y` 符号相同：用 `x - y` 的符号判断（注意溢出）
- `x + (~y) + 1` 等价于 `x - y`
- 当符号相同时，`x <= y` ⟺ `(x - y) <= 0` ⟺ `(x - y)` 的符号位为1（负数）或0（结果为0）
- 符号不同时用 `diff_sign & sign_x` 短路：x负y正则返回1

易错点:

- 正溢出问题：TMin - TMax 会变成正数，但实际上 x > y
- 负溢出问题：TMax - TMin 会变成负数，但实际上 x < y
- 符号位提取：`>> 31 & 1`，各种运算符的优先级顺序比较复杂，不清楚的时候最好就加括号

---

9. logicalNeg

题目描述

实现逻辑非运算符 `!`，即 `!x` 的功能，但禁止使用 `!` 操作符。

实现代码

```
int logicalNeg(int x) {
  /* //思路：一个数x是否为0的判断方法：x和-x的符号位都是0，当且仅当x=0 */
  int sign_x = (x >> 31) & 1;
  int sign_neg_x = ((~x + 1) >> 31) & 1;
  int temp = sign_neg_x | sign_x;
  /* //或者直接将x和-x放在一起：
  //combine_sign = (x | (~x + 1)) >> 31; */
  return ~temp + 2; /* // return temp ^ 1; 利用^1来翻转0，1 */
}
```

总操作数: 9 ops (max: 12)

解题思路

突破口/关键点:

- 利用补码的特殊性质：
  - 若 `x = 0`：则 `x = 0` 且 `-x = 0`，符号位都是0
  - 若 `x ≠ 0`：则 `x` 和 `-x` 的符号位至少有一个为1
- `sign_x | sign_neg_x` 的结果：x=0 时为0，x≠0 时为1
- `~temp + 2` 或 `temp ^ 1`：将0变成1，将1变成0

易错点:

- `~temp + 1` 等于 `-temp`，所以 `~temp + 2 = -temp + 1`
- 对于 temp=0：~0+2 = -2+2 = 0，需要再验证
- 实际上 `~temp + 2 = temp ^ 1`（异或1翻转最低位）

---

10. howManyBits

题目描述

求表示 `x` 所需的最小补码位数。

实现代码

```c
int howManyBits(int x) {
  int ans = 1; /* //符号位 */

  /* //利用符号位+算数右移构建mask */
  int mask = x >> 31;
  int next_x = 0;
  int z_mask = 0;
  int y_mask = 0;

  x = (x & ~mask) | (~x & mask);

  next_x = x >> 16;
  z_mask = ~(!next_x) + 1;
  y_mask = ~z_mask;
  x = (next_x & y_mask) | (x & z_mask); /* // x = (x >> 16) ? (x >> 16) : x */
  ans = ans + (16 & y_mask);

  next_x = x >> 8;
  z_mask = ~(!next_x) + 1;
  y_mask = ~z_mask;
  x = (next_x & y_mask) | (x & z_mask);
  ans = ans + (8 & y_mask);

  next_x = x >> 4;
  z_mask = ~(!next_x) + 1;
  y_mask = ~z_mask;
  x = (next_x & y_mask) | (x & z_mask);
  ans = ans + (4 & y_mask);

  next_x = x >> 2;
  z_mask = ~(!next_x) + 1;
  y_mask = ~z_mask;
  x = (next_x & y_mask) | (x & z_mask);
  ans = ans + (2 & y_mask);

  next_x = x >> 1;
  z_mask = ~(!next_x) + 1;
  y_mask = ~z_mask;
  x = (next_x & y_mask) | (x & z_mask);
  ans = ans + (1 & y_mask);

  z_mask = ~(!x) + 1;
  y_mask = ~z_mask;
  ans = ans + (1 & y_mask);
  return ans;
```

```
}
```

总操作数: 62 ops (max: 90)

解题思路

突破口/关键点:

- 正数：找最高有效位（MSB）的位置 + 1（加上符号位）
- 负数：找最高有效0的位置 + 1（加上符号位）
- 对于负数，转化为"找最高有效1的位置"：通过 `~x` 将问题统一

算法（二分查找）：

1. 首先处理负数：若 x < 0，则令 `x = ~x`（统一为找最高有效1）
2. 从16位开始二分查找：
   - 若 `x >> 16 != 0`，则 ans += 16，x = x >> 16
   - 否则 x 保持不变
3. 重复直到1位

易错点:

- 负数要用 `x = ~x` 而不是 `-x`（因为 -x 对 TMin 会溢出）
- 0 和 -1 需要特判：
  - 0 只需要1位（符号位）
  - -1 = 0xFFFFFFFF 需要2位（符号位 + 1）

---

11. floatScale2

题目描述

返回 `2 * f` 的位级等价表示，`f` 是单精度浮点数（unsigned int 位级表示）。

实现代码

```c
unsigned floatScale2(unsigned uf) {
  if((uf & 0x7f800000) == 0x7f800000) /* // E =11111111, NaN */
    return uf;
  /* //if(!uf) return uf; // 0 在denorm中有讨论 */
  if(((uf >> 23) << 24) == 0) /* // denorm：很巧妙的不需要讨论scale之后是denorm还是norm--操作是相同的。 */
    {
```

```
        return (uf & 0xff800000) | ((uf & 0x007fffff) << 1); /* //保留符号位和 0 阶码，尾数左移 1 位，溢出部分自动顶进阶码位 */
        /* //更节省：return (uf << 1) | (uf & 0x80000000) */
      }
    return uf + 0x00800000;
}
```

总操作数: 10 ops (max: 30)

解题思路

突破口/关键点:

- 单精度浮点数格式：1位符号 + 8位阶码 + 23位尾数
- 乘2操作：对阶码加1

三种情况处理:

1. **NaN/Inf**：`exp = 255`，直接返回原值
2. **Denorm（阶码为0）**：`frac << 1`，符号位和阶码部分不变
3. **Norm（阶码非0）**：`exp + 1`，即 `uf + 0x00800000`

易错点:

- Denorm 乘2可能变成 Norm（尾数溢出顶进阶码），但公式 `frac << 1` 恰好处理了这种情况
- 注意处理 NaN 和 Infinity（阶码全1的情况）

---

12. floatFloat2Int

题目描述

返回 `(int) f` 的位级等价表示，`f` 是单精度浮点数。

实现代码

```
int floatFloat2Int(unsigned uf) {
  int sign = uf & 0x80000000;
  int exp = (uf & 0x7f800000) >> 23;
  int E = exp - 127; /* //denorm会直接round to zero */

  int frac = uf & 0x007fffff;
  int M = frac;

  if(exp) /* //exp != 0, uf是norm */
      M = frac + 0x00800000;
  if(E >= 0)
```

```
  if(E >= 0)
  {
    if(E <= 23) M = M >> (23 - E);
    else if(E <= 30) M = M << (E - 23);
    else return 0x80000000u;
  }
  else return 0; /* // E < 0的时候, round to zero; */
  if(sign)   M = ~M + 1;
  return M;
}
```

总操作数: 15 ops (max: 30)

解题思路

突破口/关键点:

- 单精度浮点数转换：`V = (-1)^s * M * 2^E`
- **Denorm**：E = -126，M = frac/2^23，值太小直接返回0
- **Norm**：E = exp - 127，M = 1 + frac/2^23

转换步骤:

1. 提取符号、阶码、尾数
2. 计算 E = exp - 127
3. 对于 norm，构造 M = 1 + frac/2^23 = (frac + 0x00800000) / 2^23
4. 根据 E 的值移位：
   - E < 0：round to zero，返回0
   - 0 ≤ E ≤ 23：右移 `(23 - E)` 位
   - 24 ≤ E ≤ 30：左移 `(E - 23)` 位
   - E > 30：溢出，返回 0x80000000u
5. 根据符号位处理正负

易错点:

- E < 0 时结果 round to zero（返回0）
- E > 30 溢出，返回 0x80000000u
- 注意符号处理：负数用 `~M + 1` 取反

---

13. floatPower2

题目描述

返回 `2.0^x` 的位级等价表示。

实现代码

```
unsigned floatPower2(int x) {
  /* // 2.0 ^ x = 1.0 * 2 ^ (x + 127 - Bias)
    //unsigned a = 0x40000000; */
    if(x >= 0x00000080) return 0x7f800000u; /* //+inf */
    else if(x >= -126) return (127 + x) << 23; /* //norm */
    else if(x >= -149) return 0x00000001 << (x + 149); /* //denorm */
    return 0; /* //less than denorm */
}
```

总操作数: 9 ops (max: 30)

解题思路

突破口/关键点:

- `2.0^x = 1.0 * 2^x = M * 2^E`，其中 M = 1.0（尾数为0）
- 单精度浮点数：`2.0^x` 的位表示 = `(x + 127) << 23`（对于 norm）

三种情况处理:

1. **x >= 128**：阶码 >= 255，溢出到 +Inf (0x7F800000)
2. **-126 <= x < 128**：Norm 表示，`exp = x + 127`，返回 `(127 + x) << 23`
3. **-149 <= x < -126**：Denorm 表示，尾数部分为 `1 << (x + 149)`
4. **x < -149**：比最小 denorm 还小，返回 0

易错点:

- Norm 的阶码范围：-126 到 127（加上偏置127后是1到254）
- Denorm 的阶码为0，值 = `frac * 2^(-126)`
- 理解denorm的正最小值：`0x00000001 = 2^(-149)`